

Project Name: JobPilot — AI Job Application Automation Agent

Problem: Applying to jobs manually takes 2-3 hours per application. Most people either apply generically or spend hours customizing.

Solution/Define: JobPilot automates the entire job application process — given a job URL and resume, it automatically generates a tailored resume, cover letter, company research, profile match score, and interview preparation — then saves everything to Notion and emails a PDF report, all without any human involvement.

Build Agent with 2+ Tools:

<u>Tool</u>	<u>Purpose</u>
1. Webhook	Entry point — receives job URL and resume
2. Jina.ai	Scrapes job description from any URL
3. OpenAI GPT-4o-mini	Tailors resume + writes cover letter
4. Groq Llama 3.3	Company research + interview prep + match score
5. PDFShift API	Converts output to professional PDF
6. Notion API	Saves all results to a structured database
7. Gmail	Delivers PDF to user's inbox automatically

How It Works/Execution:

Once the user submits their job URL and resume through the JobPilot web form, the agent executes fully autonomously — it scrapes the job description, runs 5 AI tasks in parallel, merges results, saves to Notion, generates a PDF, and sends it via

Gmail — all without any manual steps. The entire pipeline completes in under 60 seconds.

AI Prompt Documentation:

1. Resume Tailor

Model : OpenAI · gpt-4o-mini

Prompt: JD: {jd_text - first 1000 characters}

Resume: {resume - first 800 characters}

Rewrite 5 resume bullet points that match this job perfectly.

Use keywords from the JD. Each bullet starts with a strong action verb. Keep each bullet under 20 words.

2. Cover Letter

Model : OpenAI · gpt-4o-mini

Prompt: JD: {jd_text - first 1000 characters}

Resume: {resume - first 800 characters}

Write a short cover letter in 3 paragraphs.

Sound like a real human, not a robot.

Para 1: why this job excites me.

Para 2: my best experience for this role.

Para 3: confident closing.

3. Interview QnA

Model : Groq · llama-3.3-70b-versatile

Prompt: JD: {jd_text - first 1500 characters}

Write 5 interview questions for this job with simple short answers.

Format exactly like this:

Q1: (question)

A1: (answer in 2 simple sentences) Use easy everyday language.

No jargon.

4. Company Research

Model: Groq · llama-3.3-70b-versatile

Prompt: JD: {jd_text - first 1500 characters}

From this job description tell me in simple easy words:

1. Company name and what they do (2 lines)
2. Work culture and values (2 lines)
3. Why someone would love working here (2 lines)
4. 2 smart questions to ask in the interview Write like you are explaining to a friend. Short and clear.

5. Profile Match Score

Model : Groq · llama-3.3-70b-versatile

Prompt: JD: {jd_text - first 1000 characters}

Resume: {resume - first 800 characters}

Compare this resume with the job.

Give results in simple words:

Overall Match: XX%

Skills Match: XX%

You have: (list matched skills simply)

You need: (list missing skills simply)

Experience Match: XX%

Why: (1 simple sentence) Should you apply? (Yes/Maybe/No + 1 reason in simple words)

NODE CODE LOGIC :

Text Cleaning — Code in JavaScript

1. Takes raw HTML from Jina.ai and strips all tags
2. Limits JD text to 2000 characters to save tokens
3. Removes PDF binary garbage characters from resume upload
4. Limits resume to 1500 characters
5. Passes clean jd_text, resume, and job_url to all 5 AI nodes

Learning and Challenges:

Learnings:

1. What a Webhook Actually Is:

Webhook is like entry gate of workflow who receives URL and trigger the workflow. Your automation wake up and starts working.

2. APIs function:

APIs are just like waiter/mediator who receives request from the user go to models/LLMs server who makes/process request then it get back response to the user.

3. Running Things in Parallel Saves Time:

Running things in parallel really saves time than i think. That i have used in this project. I called multiple models through APIs calling got data and merged it.

4. Garbage in Garbage out:

Till now have heard about garbage in garbage out but in this project i felt. When i gave bad prompt to the model it was not giving me desired output.

5. Token = Money:

Most interesting learning was token = money use it wisely. Because tokens are not free. In this project i have used OpenAI APIs Credits which is not free that's why in prompt I have given exact number of strings what i want in output.

And for two basic tasks used groq model APIs which is free to some extent.

Challenges:

1. JSON error:

I was trying to send JSON body to OpenAI in n8n. I kept getting invalid JSON. I learned that you can't put {{ variables }} inside JSON strings directly. You have to use JSON.stringify() to safely wrap the text. Once I understood this, every single OpenAI node worked perfectly.

2. Rate limit error : Too many tokens at once

In first attempt i hadn't specified the rate limit to the model. Who threw error that sending too much tokens at a time more than given model can receive.

Then i searched models rate limit. Specified limit in prompt and that solved my error.

3. Setup NOTION database – Page VS Database:

Setting up notion database was challenging more me. Because I was using this tool first time and it had many steps to setup from there account creation to API key generation.

Then u i had to connect that account to n8n.

Main problem came when i had to create a database in notion. I was confused between page and database there.

Fixed it after lots of attempts then understood notion well.